

# Estúdio Go2

Como o sistema está, o que ele faz hoje e para onde vai



Situação em 1 de outubro de 2026 · passos 1 a 6 concluídos · pasta `sistema/estudio/`

Para: Pedro, e quem precisar entender o projeto sem abrir o código.

# 1. Em uma página

---

O Estúdio Go2 é uma aplicação web que roda no seu computador. Dentro dela o robô quadrúpede **Unitree Go2** vive numa simulação física (**MuJoCo**) mostrada ao vivo na página. Você monta o cenário, define o que vale ponto, dá ordens ao robô por código Python e manda o estúdio repetir tentativas para achar os melhores valores — tudo pela tela, sem editar arquivos.

## O que já funciona

- **Simulação na página:** física completa a 100 Hz, vídeo a até 30 quadros/s, câmeras, modos Editar e Testar.
- **Editor de cenário:** caixas, rampas, cilindros, esferas, barras, escadas, paredes e relevo; texturas fotográficas reais (ambientCG); zonas de partida, checkpoint, chegada e proibida; **regras** que dão pontos e encerram o teste (sucesso/falha). Salvar, abrir, importar, apagar.
- **Biblioteca de ações:** ficar de pé, sentar, deitar, levantar, andar uma distância, girar um ângulo, andar/girar até parar — cada uma fechada em malha (mede e corrige) e sabendo quando terminou.
- **Programação:** editor de Python na página com o módulo `robo`; o código roda no próprio estúdio, com saída, Parar e programas gravados.
- **Treino por tentativa:** repete o programa variando valores (os que o programa declara e ajustes da marcha), pontua pelas regras do cenário, acelera a física até 20× e guarda o campeão.

## O que ainda não existe

- O robô **não aprende sozinho**: o treino de hoje ajusta números de uma marcha escrita à mão; não inventa movimentos novos. O objetivo final do projeto (seção 9) é aprendizado por reforço de verdade.
- Conversa com IA, outros modelos de robô e reconhecimento por câmera.
- Nada foi gravado no Git ainda; tudo vive na pasta do projeto (que está no OneDrive).

## Os passos

|    |                                                                                  |         |
|----|----------------------------------------------------------------------------------|---------|
| 1  | Esqueleto da página: seções, lateral recolhível, barra de estado                 | feito   |
| 2  | MuJoCo dentro da página                                                          | feito   |
| 3  | Editor de cenário, materiais, zonas, regras, salvar/importar                     | feito   |
| 4  | Biblioteca de ações com fila; dois defeitos do controlador corrigidos            | feito   |
| 5  | Programação em Python na página                                                  | feito   |
| 6  | Treino por tentativa, campeão gravado em pasta                                   | feito   |
| 7  | Aprender: aprendizado por reforço (rede neural) a partir do cenário e das regras | a fazer |
| 8  | Conversa com IA que escreve programas e chama ações                              | a fazer |
| 9  | Importar outros robôs                                                            | a fazer |
| 10 | Reconhecer com visão (câmera do robô)                                            | a fazer |

Houve uma seção "Interações" (botões da biblioteca) feita e retirada a pedido: a Programação já cobre esse uso.

## 2. Como abrir e o que precisa

```
cd "DashBoard Isaac\sistema\estudio"
python estudio.py                # abre http://localhost:8791/ no navegador
python estudio.py --aberto       # aceita outras máquinas da rede (ver aviso abaixo)
python estudio.py --sem-navegador
```

- **Python 3.14** com `mujoco` (3.11), `numpy` e `Pillow`. O servidor em si usa só a biblioteca padrão do Python — não há framework web, nem Node, nem banco de dados.
- O modelo do Go2 fica em `~/Downloads/go2_cinematica_ws/unitree_go2` (baixado do `mujoco_menagerie` na primeira vez).
- As texturas vêm do **ambientCG** (CC0, sem cadastro): dez baixadas na primeira abertura; busca e download pela página.
- Chrome, Edge ou Firefox atuais. Funciona no celular na mesma rede (layout de uma coluna).

**Aviso de segurança.** A seção Programação roda Python de verdade com os poderes do servidor. É o que ela é para ser — mas por isso só abra o estúdio na rede (`--aberto`) para quem possa rodar código nesta máquina. Por padrão ele só aceita o próprio computador.

**Duas coisas do Windows que já apareceram.** O **Smart App Control** bloqueou um plugin do MuJoCo (`sdf_plugin.dll`); o estúdio avisa e segue sem ele. O **OneDrive** segura arquivos por instantes enquanto sincroniza, o que já derrubou uma gravação; as gravações agora tentam de novo por 2 s. Como a pasta do projeto está no OneDrive, isso é um risco permanente para arquivos que mudam a cada segundo (o treino grava a cada tentativa).

### Pastas

| PASTA                    | O QUE GUARDA                                                                                                  |
|--------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>simulacoes/</code> | cada cenário como <code>.json</code> (objetos, zonas, regras, materiais) e uma miniatura <code>.jpg</code>    |
| <code>programas/</code>  | os programas em Python, um <code>.py</code> por nome — abrem em qualquer editor                               |
| <code>treinos/</code>    | cada treino como <code>.json</code> : definição, cópia do cenário e do código, todas as tentativas, o campeão |
| <code>materiais/</code>  | texturas baixadas e enviadas, com créditos                                                                    |
| <code>painel/</code>     | a página: HTML, CSS e JavaScript, sem bibliotecas externas                                                    |

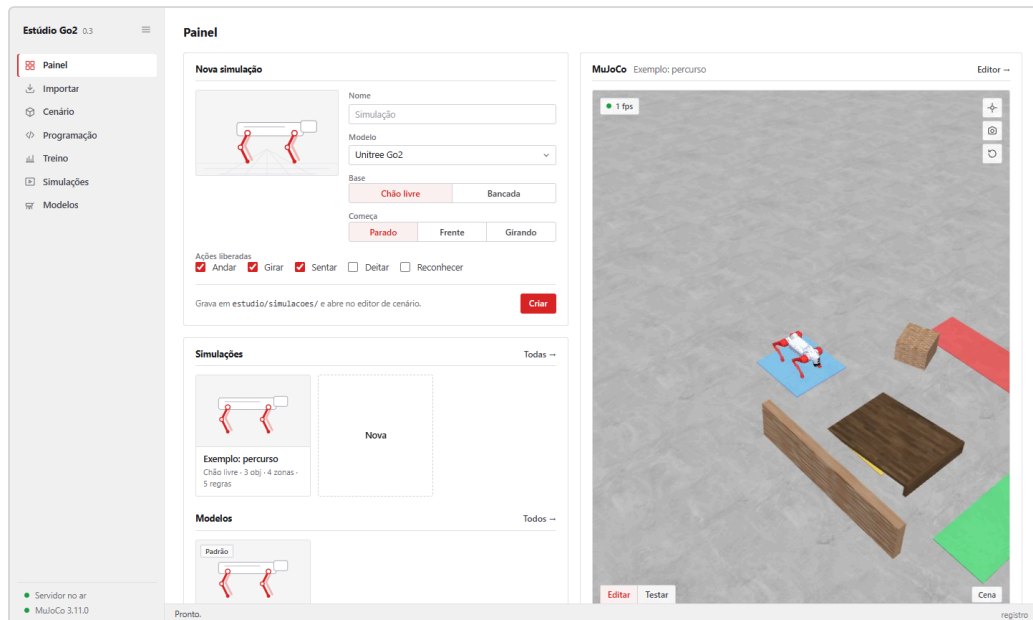
Hoje a pasta de simulações tem só o "Exemplo: percurso" (criado para este relatório); os programas, só o "Exemplo percurso".

### 3. As seções da página

Página branca, vermelho só onde precisa chamar atenção. A lateral recolhe; abaixo de 640 px vira barra de abas. A barra de baixo mostra a última mensagem e guarda um registro.

#### Painel

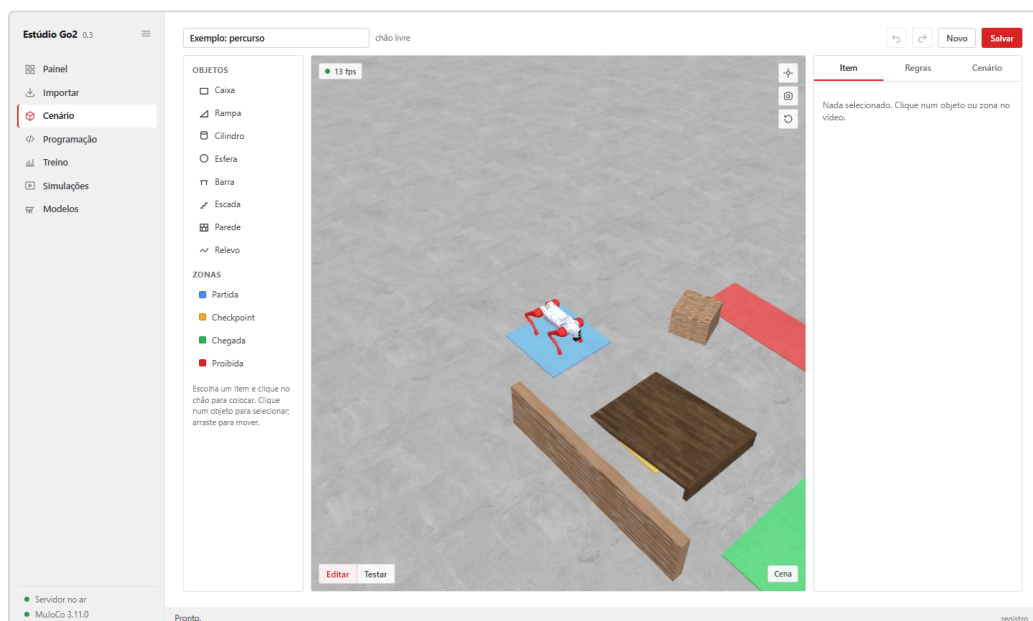
Cria uma simulação (nome, base chão livre ou bancada, como começa, ações liberadas) e mostra o MuJoCo ao vivo com o cenário aberto. Galerias resumidas de simulações e modelos.



Painel: formulário de nova simulação à esquerda, MuJoCo ao vivo à direita.

#### Cenário (o editor)

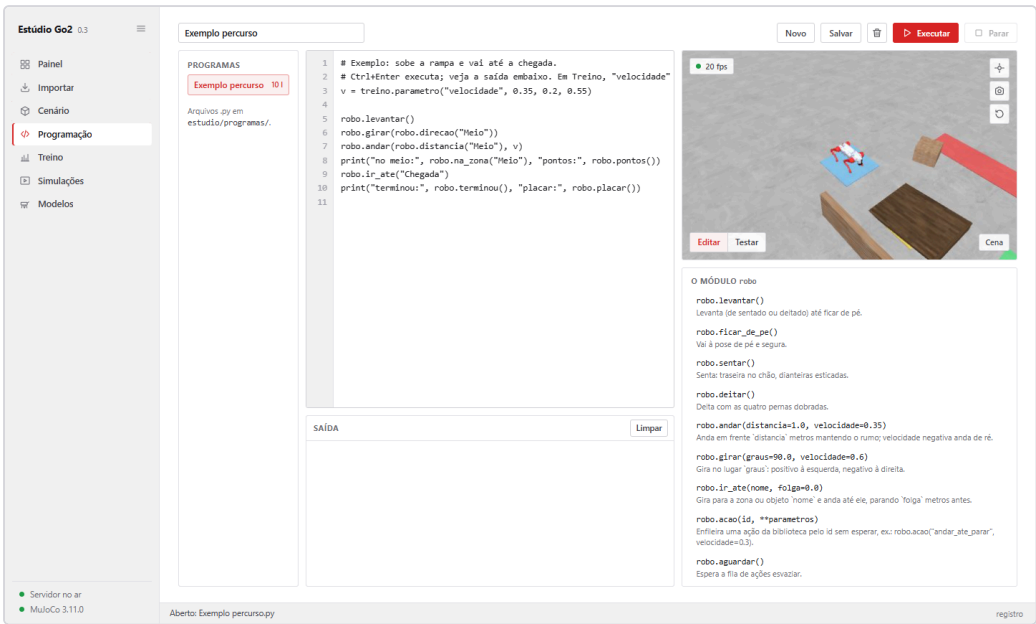
Paleta à esquerda, vídeo no meio, propriedades à direita. Clique no chão coloca; clique no objeto seleciona; arraste move. Desfazer/refazer, atalhos de teclado, salvar com miniatura.



"Exemplo: percurso": partida (azul), rampa, caixa, parede, checkpoint, chegada (verde) e zona proibida (vermelha).

## Programação

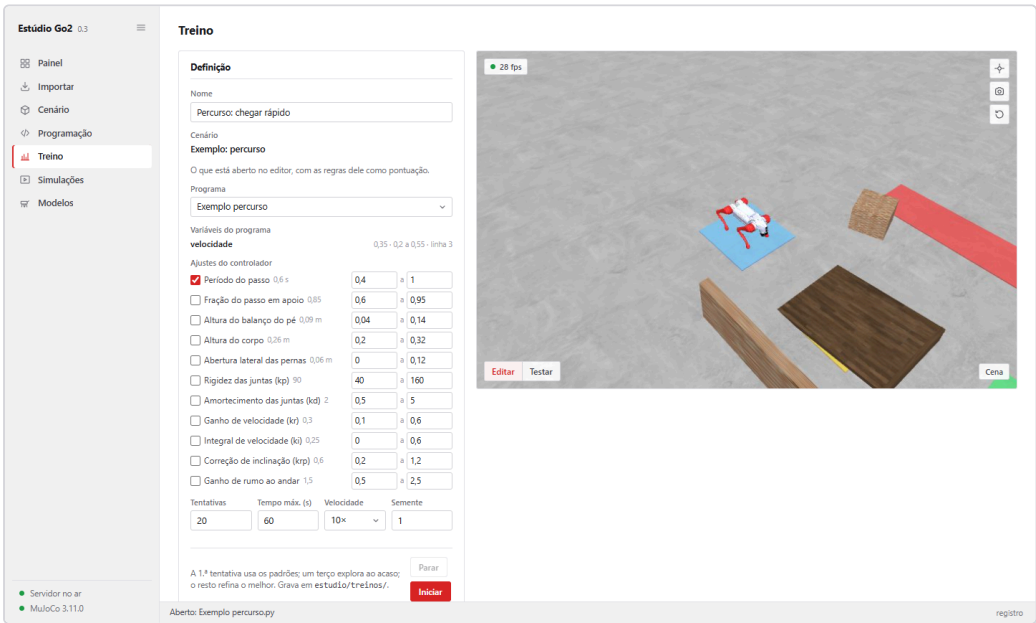
Editor de código com numeração, lista de programas gravados, saída embaixo (com o tempo do teste em cada linha), MuJoCo ao lado e a referência do módulo `robo` — clicar numa função insere no código.



Programação com o "Exemplo percurso" aberto.

## Treino

Definição à esquerda (cenário aberto, programa, variáveis, orçamento), MuJoCo e andamento à direita: tentativa atual, campeão, tabela com barras.



Treino: as variáveis do programa são lidas do código; os ajustes do controlador são marcados na lista.

## Importar · Simulações · Modelos

Importar recebe uma simulação `.json` de outro estúdio e texturas (busca no `ambientCG` ou um `.zip/imagem` sua). Simulações lista tudo o que está gravado, com abrir, testar e apagar. Modelos mostra os robôs — só o Go2 por enquanto.

## 4. Como funciona por baixo

|                                                   |                                                             |                                                           |                                                           |                                                                 |
|---------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------|
| <b>Página</b><br>HTML + JS puro;<br>pede e mostra | <b>Servidor</b><br>Python, biblioteca<br>padrão, porta 8791 | <b>Mundo</b><br>a cena, a física, o<br>placar, as câmeras | <b>MuJoCo</b><br>física a 100 Hz,<br>desenho fora da tela | <b>Disco</b><br>simulações,<br>programas, treinos,<br>materiais |
|---------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------|

- **Vídeo:** o servidor desenha a cena fora da tela e manda um JPEG atrás do outro (MJPEG) para uma `<img>` comum. Só desenha quando alguma tela está visível. Nesta máquina (Intel UHD) um quadro custa ~22 ms sem sombras — por isso as sombras ficam desligadas e o vídeo mira 30 quadros/s.
- **Física:** uma thread própria avança a simulação em passos de 10 ms, em tempo real; no treino, vários passos por período (até 20×). Outra thread desenha. As duas compartilham uma trava.
- **Cena:** o cenário vira um XML do MuJoCo gerado na hora. Mudar posição, tamanho ou material de um objeto é ajustado ao vivo; mudar a estrutura (objeto novo, relevo) recompila em ~1 s. Todos os arquivos (malhas do robô, texturas) entram pela memória, porque o MuJoCo não abre pastas com acento no Windows ("Área de Trabalho").
- **Clique no vídeo:** a página manda a posição relativa; o servidor converte em um raio da câmera e descobre objeto, zona ou ponto do chão.
- **Estado:** a página pergunta `/api/estado` a cada segundo (modo, cenário, ação em curso, fila, placar, programa, velocidade) e redesenha o que mudou.

### Arquivos do servidor

| ARQUIVO                      | PAPEL                                                                              |
|------------------------------|------------------------------------------------------------------------------------|
| <code>estudio.py</code>      | o servidor HTTP: entrega a página, o vídeo e todos os endereços da API             |
| <code>mun</code>             | a cena, a física, a fila de ações, o placar (regras), o desenho, o apontar         |
| <code>cenario.py</code>      | formato do cenário, validação, geração do XML, disco                               |
| <code>materiais.py</code>    | texturas do ambienteCG (fábrica, busca, download) e enviadas                       |
| <code>acoes.py</code>        | a biblioteca de ações e a validação dos parâmetros                                 |
| <code>controle.py</code>     | o controlador: marcha "crawl" com PD nos torques (copiado da bancada e corrigido)  |
| <code>programa.py</code>     | executa o código do usuário com os módulos <code>robo</code> e <code>treino</code> |
| <code>treino.py</code>       | o treino: variáveis, busca, tentativas, campeão, pasta <code>treinos/</code>       |
| <code>mujoco_carga.py</code> | importa o MuJoCo tolerando o plugin bloqueado pelo Windows                         |
| <code>LEIAME.md</code>       | a documentação técnica completa, incluindo a tabela de endereços da API            |

## 5. Cenário e regras

O cenário é um `.json` com chão (material, atrito, elasticidade), objetos, zonas e regras. É o que o usuário constrói; o estúdio não traz tarefa pronta.

| PARTE     | O QUE É                                                                                                                                                                                                                  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Objetos   | caixa, rampa, cilindro, esfera, barra, escada, parede e relevo (campo de alturas). Cada um com posição, tamanho, giro, material, atrito, elasticidade e se é fixo ou solto (física completa, pode ser empurrado).        |
| Materiais | texturas fotográficas reais do <code>ambientCG</code> (concreto, asfalto, madeira, grama, terra, borracha, metal, azulejo, tijolo, tapete e o que você buscar). O brilho vem do mapa de rugosidade.                      |
| Zonas     | retângulos no chão, sem colisão: partida (onde o robô nasce), checkpoint, chegada, proibida.                                                                                                                             |
| Regras    | <i>quando</i> acontece algo → <i>pontos</i> e, opcionalmente, <i>fim</i> (sucesso ou falha). Gatilhos: entrar/sair de zona, tocar objeto, cair, passar de N segundos, chegar a N pontos. "Uma vez" evita contar de novo. |

No modo **Testar** o placar avalia as regras a cada passo da física: pontos, tempo, eventos recentes e, quando uma regra encerra, o resultado. É esse placar que o treino usa como nota.

### Exemplo gravado: "Exemplo: percurso"

- Rampa de madeira a 1,6 m, caixa solta de tijolo fora do caminho, parede ao lado.
- Zonas: partida na origem, checkpoint "Meio" sobre a rampa, "Chegada" a 3 m, "Fora" (proibida) ao lado.
- Regras: entrar no Meio +50; entrar na Chegada +100 e sucesso; entrar em Fora -100 e falha; cair -30 (cada vez); passar de 90 s, falha.

## 6. Biblioteca de ações e o controlador

Uma ação é algo que o robô sabe fazer, sabe quando terminou e se deu certo. O mundo roda uma por vez, em fila; Parar limpa a fila. Botões, programas, treino e (depois) a IA passam pela mesma fila.

| AÇÃO                             | PARÂMETROS                         | O QUE FAZ                                                                                          |
|----------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------|
| ficar_de_pe, levantar            | —                                  | leva as juntas à pose de pé numa curva suave                                                       |
| sentar, deitar                   | —                                  | vai à pose e segura; falha se tombar                                                               |
| andar                            | distância (m),<br>velocidade (m/s) | anda em frente mantendo o rumo; velocidade negativa anda de ré; termina ao assentar dentro de 6 cm |
| girar                            | graus, velocidade<br>(rad/s)       | gira no lugar segurando a posição; termina dentro de 5°                                            |
| andar_ate_parar, girar_ate_parar | velocidade                         | segue até receber Parar                                                                            |

O controlador é uma marcha "crawl" com controle PD nos torques das 12 juntas, escrito à mão (copiado da bancada de cinemática que já existia). No passo 4 dois defeitos herdados foram corrigidos: a velocidade angular estava no referencial errado (o robô só ficava de pé andando para +x — era a causa das quedas que você viu nos primeiros testes) e o integrador empurrava o robô 15 cm para trás ao parar. Hoje a cadeia sentar → levantar → andar 1 m → girar 90° → andar 1 m → deitar → levantar roda sem queda em qualquer rumo, com erro final de posição em torno de 25 cm — que é o que o treino melhora.

**Limite importante.** A marcha é um conjunto de equações com uns 20 números (período do passo, altura do corpo, rigidez das juntas...). Qualquer treino sobre ela só mexe nesses números. Ela não sobe degrau alto, não corre, não se recupera de uma queda de lado — não há movimento para isso dentro dela.

## 7. Programação

O código roda numa thread do próprio servidor. O objeto `robo` enfileira ações e **espera cada uma terminar**, devolvendo `True` se deu certo. `print` vai para o painel de saída. **Parar** interrompe a ação em curso e o programa na linha seguinte — funciona até num `while True` sem robô. Erros aparecem com a linha marcada. Executar liga o modo Testar sozinho.

```
v = treino.parametro("velocidade", 0.35, 0.2, 0.55) # padrão 0,35; o treino varia de 0,2 a 0,55

robo.levantar()
robo.girar(robo.direcao("Meio"))          # quanto girar para apontar à zona
robo.andar(robo.distancia("Meio"), v)
print("no meio:", robo.na_zona("Meio"), "pontos:", robo.pontos())
robo.ir_ate("Chegada")                    # gira e anda até a zona
print("terminou:", robo.terminou(), "placar:", robo.placar())
```

### GRUPO    FUNÇÕES

|         |                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mover   | <code>levantar()</code> <code>ficar_de_pe()</code> <code>sentar()</code> <code>deitar()</code> <code>andar(distancia, velocidade)</code> <code>girar(graus, velocidade)</code> <code>ir_ate(nome, folga)</code> <code>acao(id, **parametros)</code> <code>aguardar()</code> <code>parar()</code> <code>esperar(segundos)</code> <code>reiniciar()</code> <code>ajustar(**valores)</code> <code>ajustes()</code> |
| Medir   | <code>posicao()</code> <code>altura()</code> <code>rumo()</code> <code>postura()</code> <code>tempo()</code> <code>quedas()</code> <code>placar()</code> <code>pontos()</code> <code>terminou()</code>                                                                                                                                                                                                          |
| Cenário | <code>zonas()</code> <code>objetos()</code> <code>zona(nome)</code> <code>objeto(nome)</code> <code>na_zona(nome)</code> <code>distancia(nome)</code> <code>direcao(nome)</code>                                                                                                                                                                                                                                |
| Treino  | <code>treino.parametro(nome, padrao, minimo, maximo)</code> — fora do treino vale o padrão                                                                                                                                                                                                                                                                                                                      |

Programas são gravados em `programas/<nome>.py`. O texto do editor sobrevive a recarregar a página.

## 8. Treino por tentativa — o que ele é e o que não é

É **busca de parâmetros com medição** ("jeito 2", sem rede neural). Você define tudo; o estúdio só repete e mede.

1. O robô volta à partida e o placar zera.
2. O estúdio escolhe valores para as variáveis: as que o programa declara com `treino.parametro` e os ajustes do controlador que você marcou (período do passo, altura do corpo, rigidez das juntas, ganho de rumo...), dentro dos limites.
3. Roda o programa com esses valores, a até 20× o tempo real. A tentativa termina quando o teste termina (regra com fim), o tempo máximo esgota ou o programa acaba.
4. Anota pontos, fim, tempo, quedas e valores — e grava no `.json` na hora.

**Como escolhe:** a 1.<sup>a</sup> tentativa usa os padrões (a referência "como era"); um terço explora ao acaso dentro dos limites; o resto perturba o melhor até ali com passos cada vez menores. Ordem: mais pontos, depois menos quedas, depois menos tempo. Semente fixa: o mesmo treino repete igual.

### O que você tem de garantia

- O campeão é **reproduzível**: o MuJoCo é determinístico. "Ver o campeão" repete a tentativa vencedora.
- O campeão **nunca é pior** que os padrões: se nada superar a 1.<sup>a</sup> tentativa, o campeão é ela.
- Tudo fica no arquivo, tentativa por tentativa.

### O que não tem

- "Melhorou" é **na sua pontuação, no seu cenário**. Régua frouxa, melhora frouxa.
- Não generaliza: campeão no concreto não é campeão na grama.
- Poucas tentativas podem não achar o melhor.
- Não inventa movimento: o teto é o da marcha.

### Reutilizar o campeão (hoje só se você mandar)

- **Ver o campeão**: roda a melhor tentativa de novo, a 1×, no cenário gravado.
- **Gravar programa campeão**: escreve um `.py` com os valores vencedores no lugar dos padrões e, se houver, `robo.ajustar(...)` no início.
- **Usar ajustes**: aplica ao controlador — mas só até reiniciar o estúdio.

Teste de aceite do passo 6: andar até uma chegada a 2 m com velocidade, distância e período do passo variando; 6 tentativas em 4 s reais (20×); campeão = a mais rápida com 150 pontos; programa campeão gerado; ajustes restaurados ao fim.

## 9. Para onde vai: o robô aprender de verdade

---

O objetivo final acordado: *"o robô aprenda sozinho repetidamente depois de uma tarefa bem construída e um cenário bem construído, repetindo os movimentos até concluir a tarefa e aprendendo sozinho o melhor método"*. Isso é **aprendizado por reforço** (RL): uma rede neural recebe o estado do robô e decide os torques das 12 juntas a cada instante; começa sem saber nada e aprende a partir só da pontuação, em milhares de tentativas por minuto, em várias cópias da simulação em paralelo. É assim que o Go2 real anda hoje, e o MuJoCo é a ferramenta usada para isso.

O que já existe serve de base: o cenário editado vira o ambiente, as regras viram a recompensa, o MuJoCo na página vira a janela para ver o robô aprendendo, e a política treinada vira uma ação da biblioteca (chamável por programas, pelo chat com IA e por treinos seguintes).

### O que muda e o que custa

- Entra o **PyTorch** (200 MB a 2 GB) — inevitável, é a rede neural.
- Tempo de treino passa de segundos para **horas** por habilidade (ficar de pé e andar até a zona: algumas horas nesta máquina; subir escada: uma noite ou mais). O computador fica ocupado, esquenta e a ventoinha sobe — sem risco de dano; na tomada, com ventilação livre.
- **A recompensa é tudo.** Regra só "chegar na zona" produz robô que chega arrastando a barriga. Entram regras padrão sugeridas (postura ereta, pouco torque, pouca trepidação) que você liga/desliga.
- O primeiro resultado decente demora: montar ambiente, rede, laço, acertar a recompensa leva rodadas, e cada rodada custa horas. Passos maiores do que os de até agora.
- Funciona na simulação; levar ao robô físico é outra etapa ("sim-to-real").

### Plano proposto

- |    |                                                                                                                                                                                                                                                                              |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7  | <b>Aprender</b> , em sub-passos testáveis: (a) ambiente e recompensa a partir do cenário, com um robô aleatório; (b) o laço aprendendo a ficar de pé; (c) andar até a zona; (d) a política como ação da biblioteca, gravada em pasta, com parar/continuar e a curva na tela. |
| 8  | Conversa com IA (Claude por API ou modelo local — a escolher) que escreve programas e usa as políticas aprendidas.                                                                                                                                                           |
| 9  | Importar outros robôs.                                                                                                                                                                                                                                                       |
| 10 | Reconhecer com visão.                                                                                                                                                                                                                                                        |

### Decisões em aberto

- Instalar o PyTorch e começar o passo 7.
- Qual IA no passo 8: Claude por API (chave, melhor qualidade) ou modelo local (Ollama, sem custo, instalar algo).
- Gravar os passos 1–6 no Git — nada foi commitado ainda, e o volume de trabalho só cresce.
- Opcionalmente, antes do 7: memória permanente dos ajustes (`ajustes.json` carregado ao abrir), treino partindo do campeão anterior e a curva de pontuação por cenário — pequenos, e fazem o treino de hoje acumular em vez de só medir.